

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

Client: Christophe Arnaud (MicroOmiX)

Supervisors: Norbert Dumas and Jean Dellinger

Group members :

PIERRARD Antoine (Project manager)

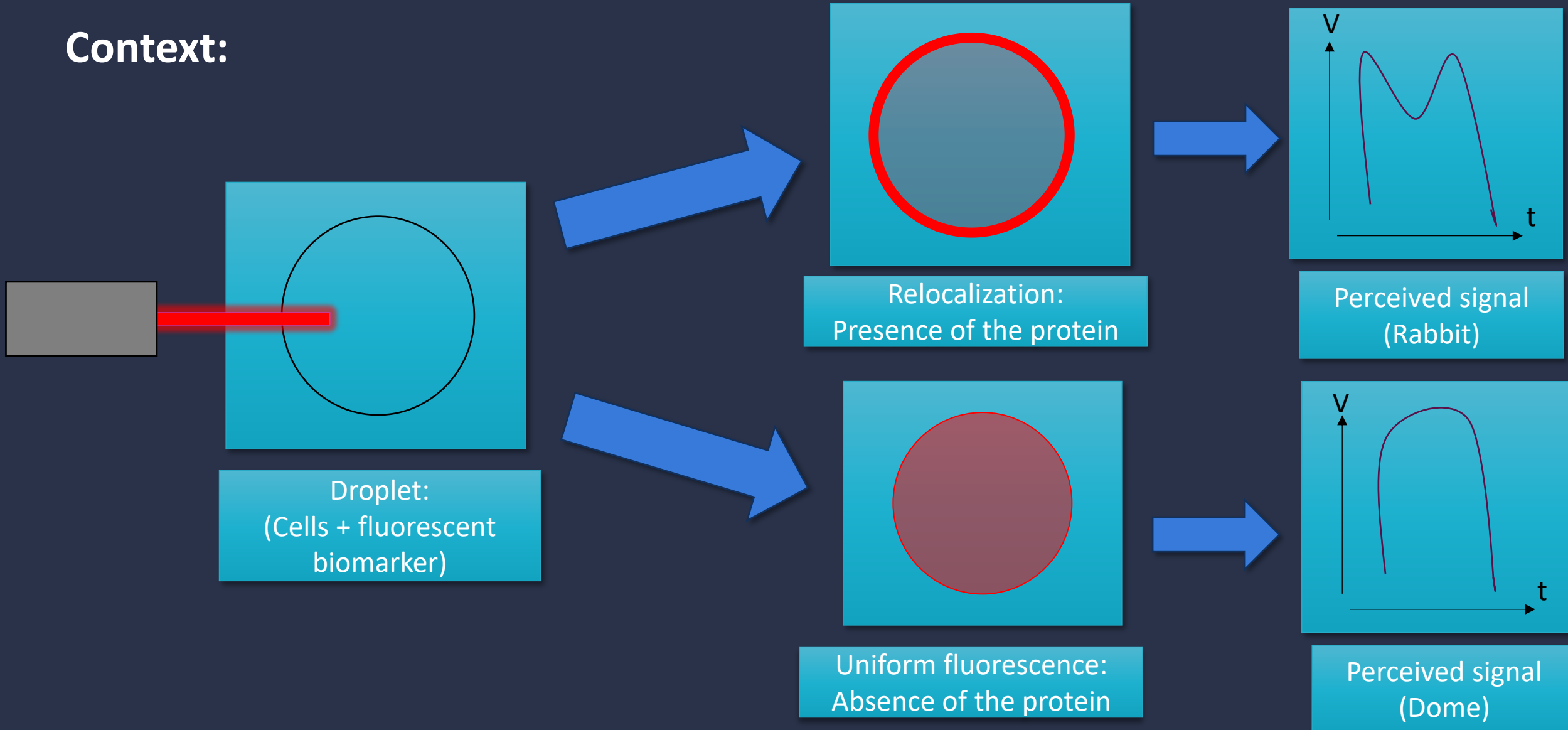
MARLIERE Clara

MANZ Arthur

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

1

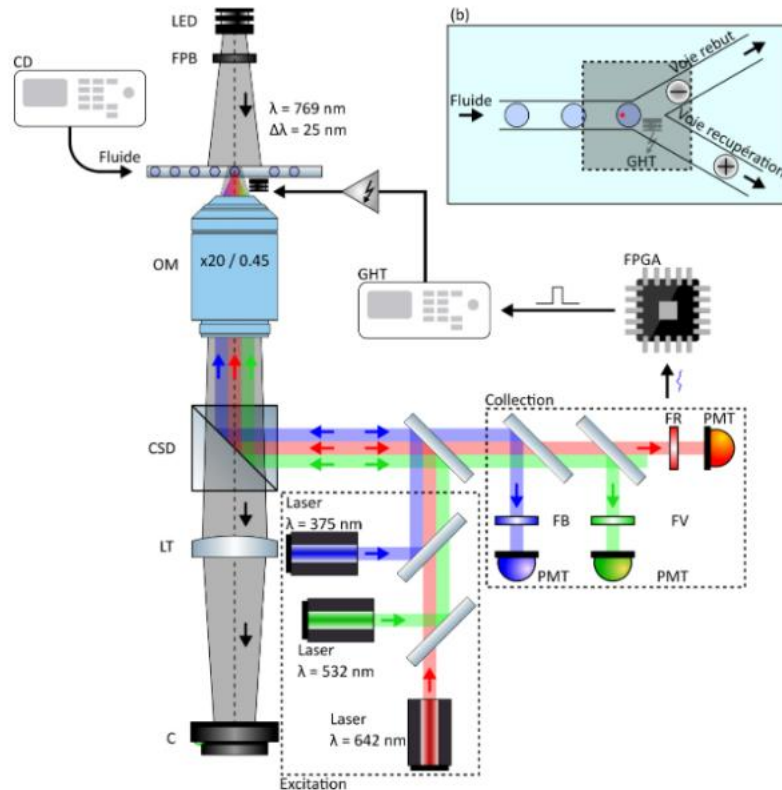
Context:



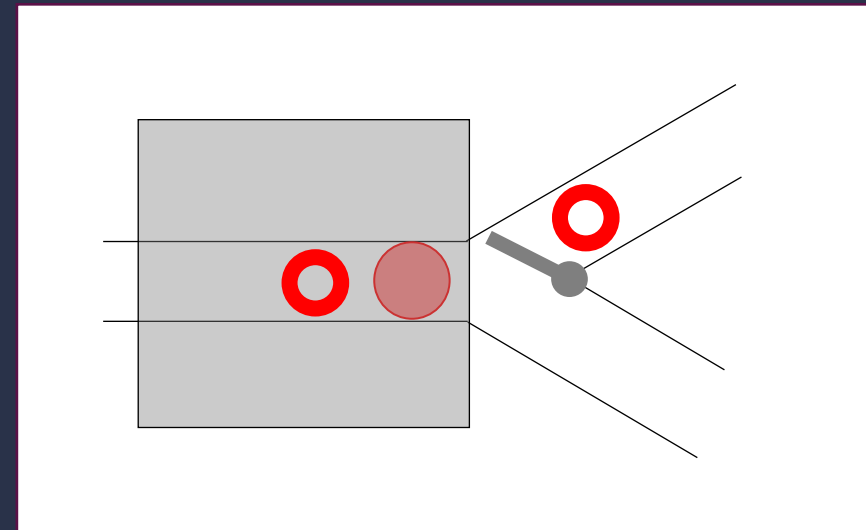
TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

2

Operation:



Detailed system diagram

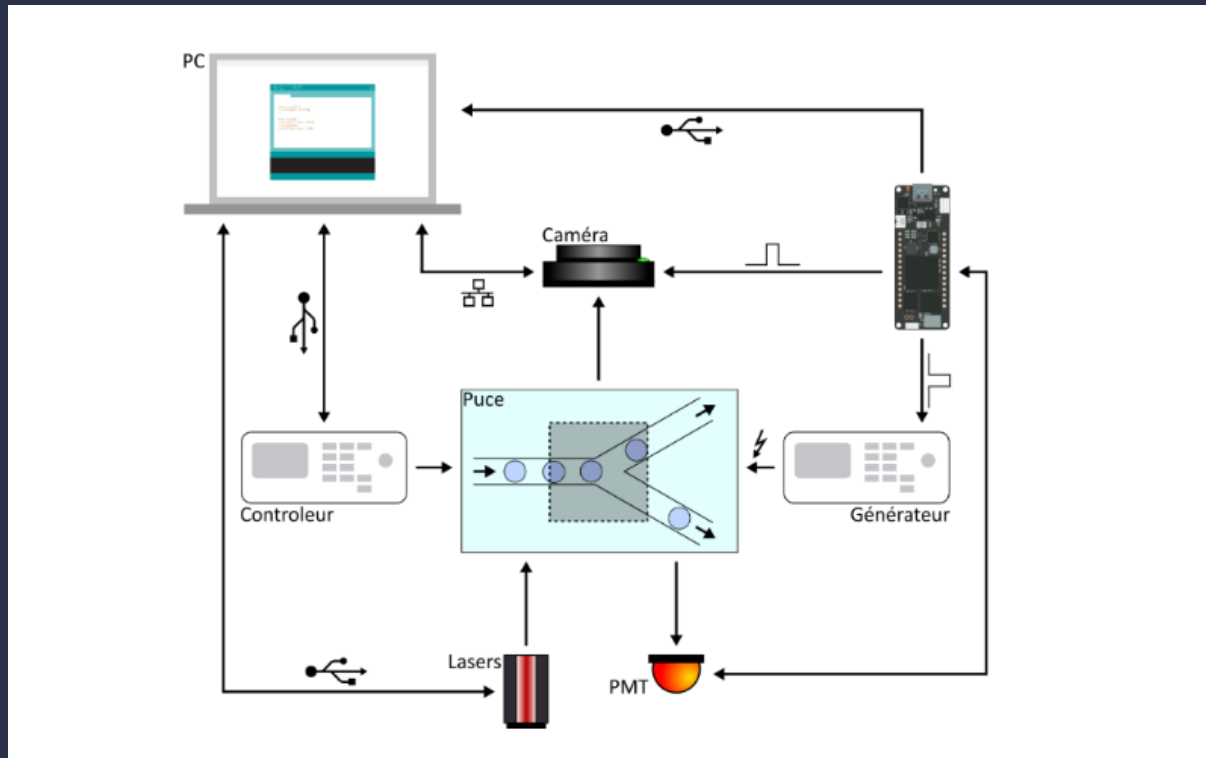


Sorting platform
throughput: 200 droplets/s

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

3

Detailed operation of the part that concerns us:



Problem statement:
How can these droplets be sorted?

Principle diagram of the optofluidic system from MicroOmiX. The components of the setup and the data exchanges between them are represented.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

4

Previous work:

MicroOmiX (client) Red Pitaya LabVIEW	TI2A project 2020-2021: Red Pitaya -> Arduino Python interface	Final-year internship: Hugo Tessoneaux Arduino LabVIEW
- Invention of the high-throughput cell screener	- signal sorting algorithm (quite innovative) - Python interface - Implementation of the processor architecture	- Optimization of the processor architecture - LabVIEW interface - Draft and partly dysfunctional code

Constraints
- process droplets at a throughput of 200 droplets per second - 200 kHz / 16-bit sampling

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

5

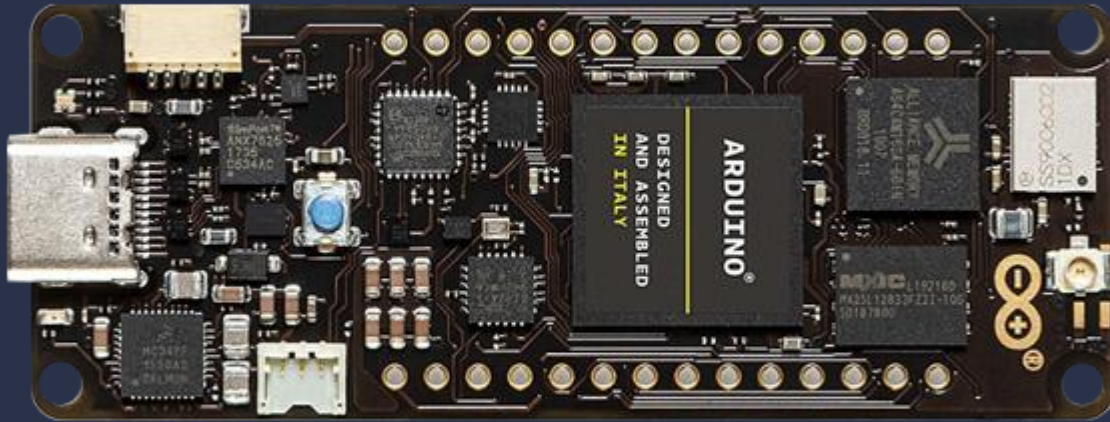
Objective: Sort the droplets

Requirements	Physical resources	Documentary resources
- throughput: 100-200 droplets/second - sampling frequency: 200 kHz / 16 bits - Control through LabVIEW - Microcontroller: Portenta H7	- The original machine (Faculty of Pharmacy, Illkirch campus) - 2 Arduino Portenta H7 - A LabVIEW license on several TPS computers - A trial version (valid for 2 months) on our personal computers.	- Final report from the TI2A students - code Arduino - Python interface code - Hugo's final-year internship report - code Arduino - Python code

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

6

Selection of processor functions:

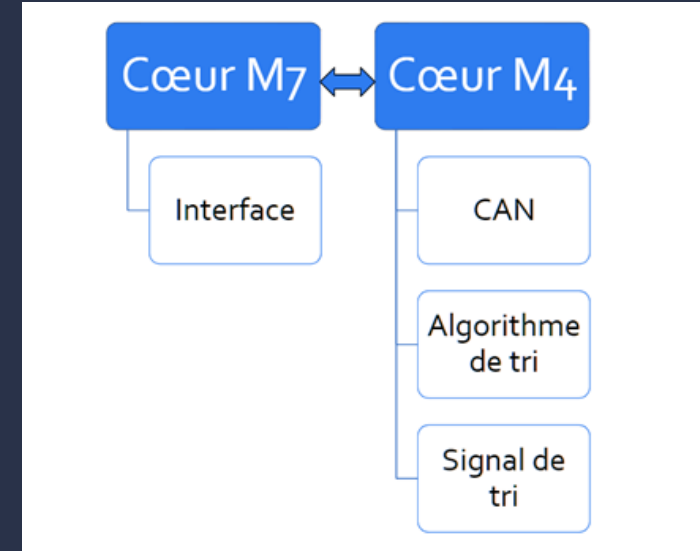


Arduino Portenta
H7

32-bit dual-core processor:

- M4: 240 MHz
- M7: 480 MHz

Three 16-bit ADCs



Why this layout?

The serial port (link with LabVIEW) is on the M7 processor

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

7

Work carried out before the 1st presentation:

Action	Method
Reading the documentation	• Each report is more than 70 pages long and full of technical terms, but understanding them is essential for the next steps
Learning how the Arduino works: (Most of us started with little or no knowledge)	• Training at the FabLab with Mr Dumas • Online tutorials using kits loaned by the FabLab
Getting started with LabVIEW: (We started with no knowledge)	• Very instructive classes and tutorials taught by Mr Dellinger.
Familiarization with our predecessors' programs	• Added comments to each of their functions/blocks to make them clearer.

Work carried out after the 1st presentation:

Action	Method
Understanding Hugo's Arduino code	• Documentation: - Portenta H7 specifications (ports, maximum voltage and current, etc.) - Commands for the processor registers (STM32H747XI) - Arduino forums - Annotation of each line of code
Isolation, debugging and testing of each acquisition-code function of data.	• Implementation of interrupt-handling routines to know the state of each component. • Test of the sampling frequency and resolution using a function generator
Routing data from one core to the other through shared memory	• Reflection on the method used to route information (FTDI, USB)
Sorting algorithm	• Adaptation of Hugo's code

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

8

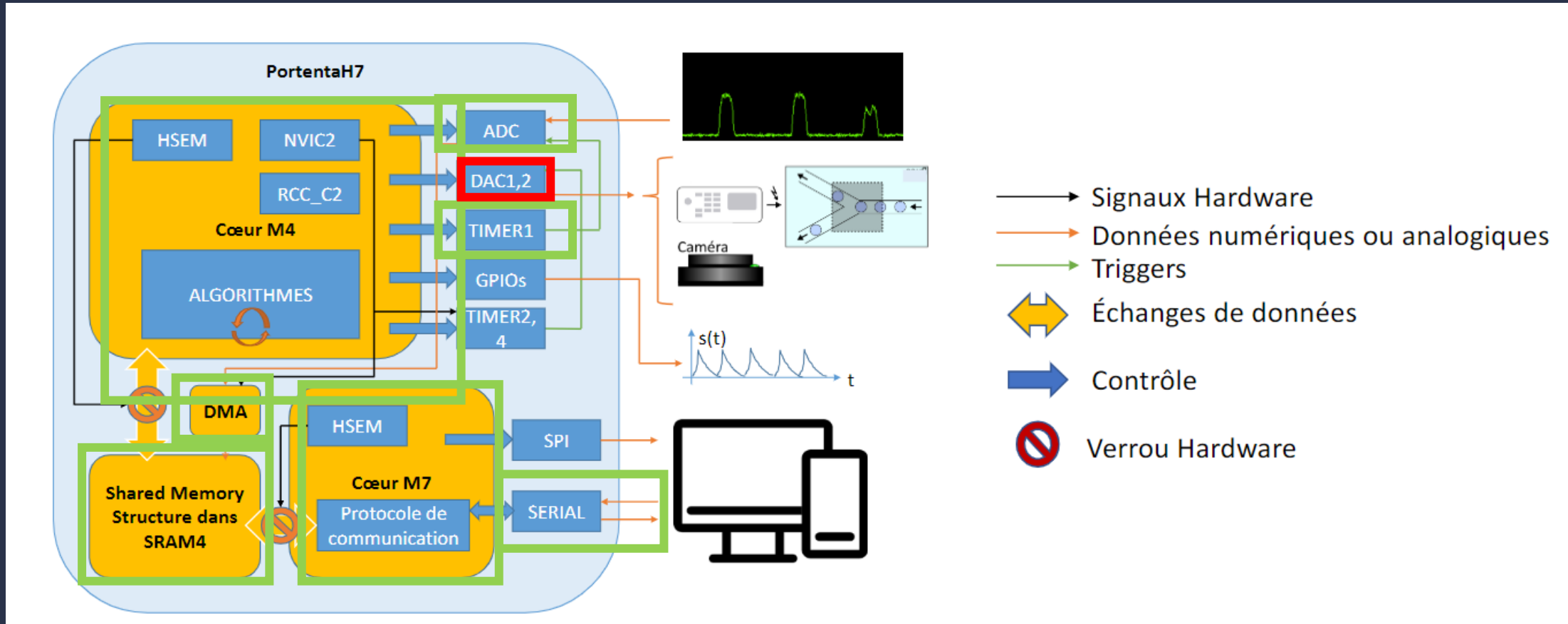
Work carried out after the 2nd presentation:

Action	Method
Improvement of data transfer from Arduino to LabVIEW.	• Use of Serial.write() • Increase of the Baud rate • Implementation of the decomposition (Arduino)/reconstruction(LabVIEW) of 16-bit signals into bytes.
Testing the sorting algorithm	• Generation of model signals using a function generator • Verification of the values calculated by the Portenta: droplet size, SFR
Data storage	• Implementation of a CSV-based storage system for SFR and ADC data.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

9

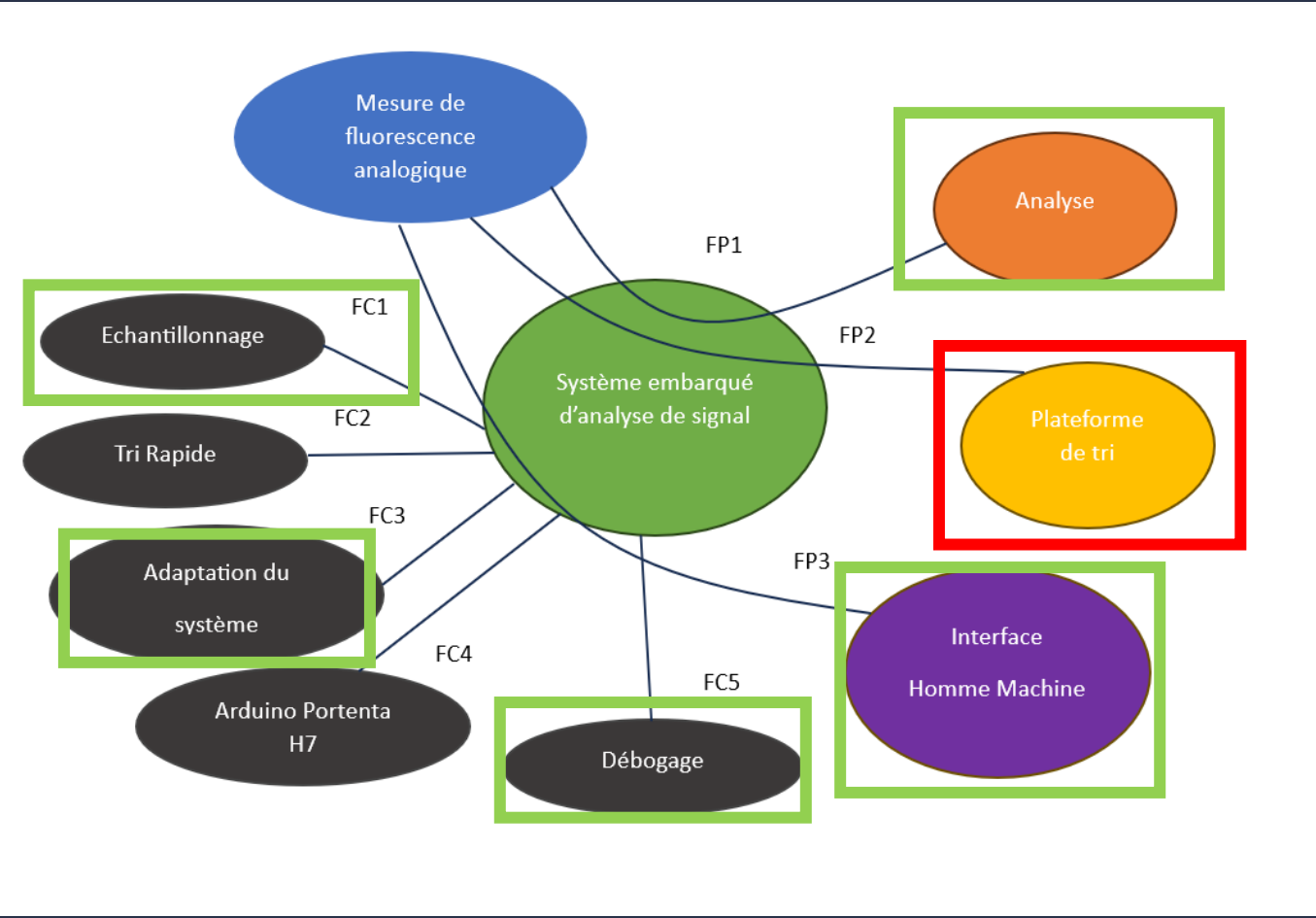
Detailed role of the Portenta H7 in the system:



The main problem is that the functions provided by Arduino to manage the ADC (AnalogRead()) do not make it possible to exceed acquisition frequencies of 17 kHz

To reach the 3.3 MHz promised by the ADC, our predecessors wrote their own functions, which we had to understand and isolate.

Functional analysis



Associated specifications

Id		Fonction	Flexibilité 0	Flexibilité 1
FP1	Analyser	A partir des signaux acquis, Déterminer s'il s'agit d'une « cloche », ou d'une « oreille de lapin » par le critère de SFR (Simple Fluorescence Ratio).	Utilisation d'autres méthodes d'analyses	
FP2	Trier	Générer un signal carré analogique d'amplitude 0/2V de 15kHz avec rapport cyclique modifiable		
FP3	Contrôler Et Visualiser	Faire une interface graphique sur l'ordinateur Permettant : - de visualiser le signal en temps réel - de régler les seuils de début de mesure - d'afficher un histogramme des tailles des gouttes et des SFR. - identifier les gouttes triées (envoi d'un signal de déclenchement de prise de vue caméra et visualisation du signal sur l'interface utilisateur)		
FC1	Échantillonner	Dynamique : 16 bits Fréquence 100kHz	Dynamique : 12 bits Fréquence 200kHz	
FC2	Trier rapidement	5 ms entre 2 cellules (200 gouttes par seconde)	1 ms entre 2 cellules (1000 gouttes/s)	
FC3	S'adapter au signal	Largeur variable des cellules : nécessite la création d'un seuil adapté afin de bien détecter leur taille		
FC4	Arduino Portenta H7	Carte récente avec peu de documentation. Pour échantillonner à haute fréquence, il faut directement travailler avec la mémoire.		
FC5	Adopter une approche modulaire	Découper le code Arduino et LabVIEW en blocs que nous validerons indépendamment		

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

11

Specifications in terms of priority (client)

Category	Tasks to complete	Priority
Portenta H7	-Sampling frequency at 200 kHz / 16 bits	1
	-Send a trigger to the camera	2
	-Send a trigger to the generator	1
	-Understand and use communication protocols	1
	-Communication Inter-core	1
LabVIEW	-Display ADC data	1
	-Highlight population distribution	2
	-Choose data-processing methods	3
	-Possibility of saving data	1
	-Real-time display of the distribution of sorted droplets.	2
Algorithms	-SFR calculation algorithm	1
	-Rabbit-ear identification algorithm	1

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

12

Timer

```
TIM1->PSC = 47;
```

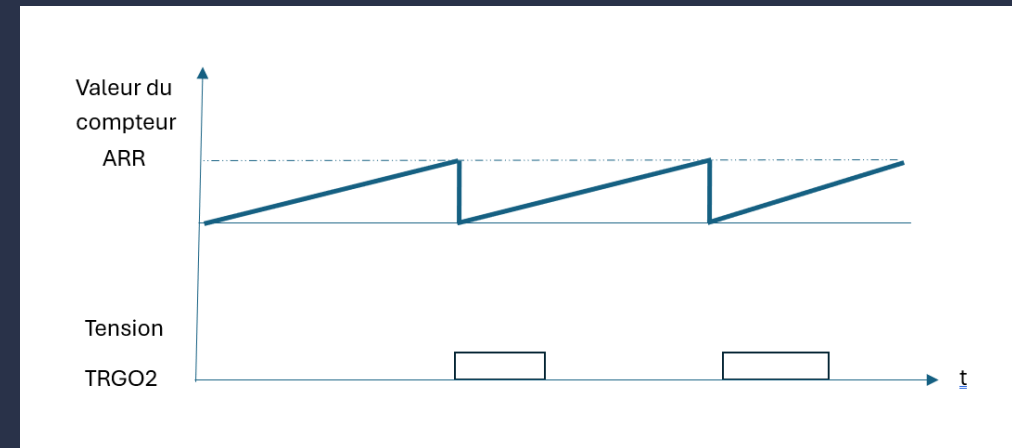
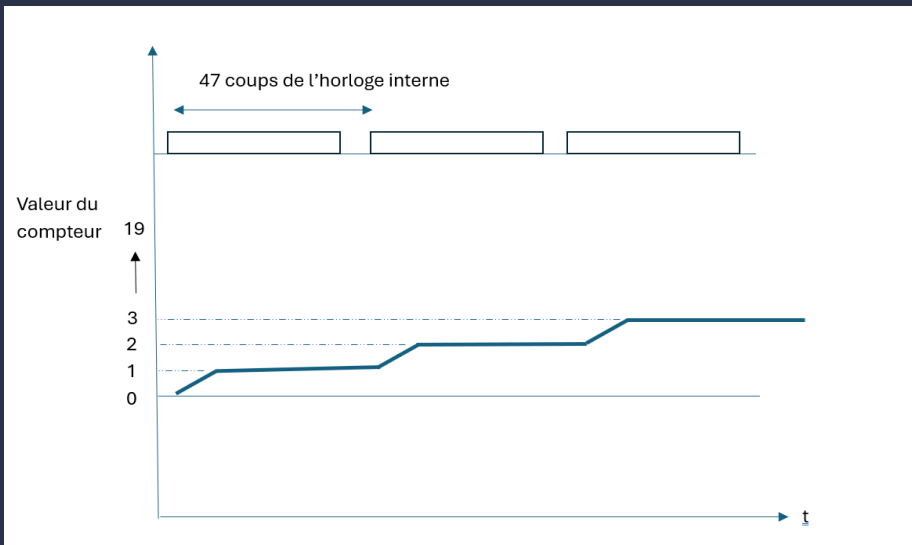
Prescaler: increments the counter tous 47 clock ticks

```
TIM1->ARR = 19;
```

Auto Reload Register (ARR) : Resets the counter to 0 when the counter **exceeds** 19 (ARR).

```
SET_BIT(TIM1->CR2, TIM_CR2_MMS2_1);
```

Triggers a signal on TRGO2 each time the counter reaches ARR



$$f_{timer} = \frac{f_{coeur\ M4}}{PSC * (ARR + 1)} = \frac{240 * 10^6}{47 * 20} \approx 250\ KHz$$

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

13

ADC (Analog to Digital Converter)

```
CLEAR_BIT(ADC1->DIFSEL, ADC_DIFSEL_DIFSEL_13);  
SET_BIT(ADC1->PCSEL, ADC_PCSEL_PCSEL_13);
```

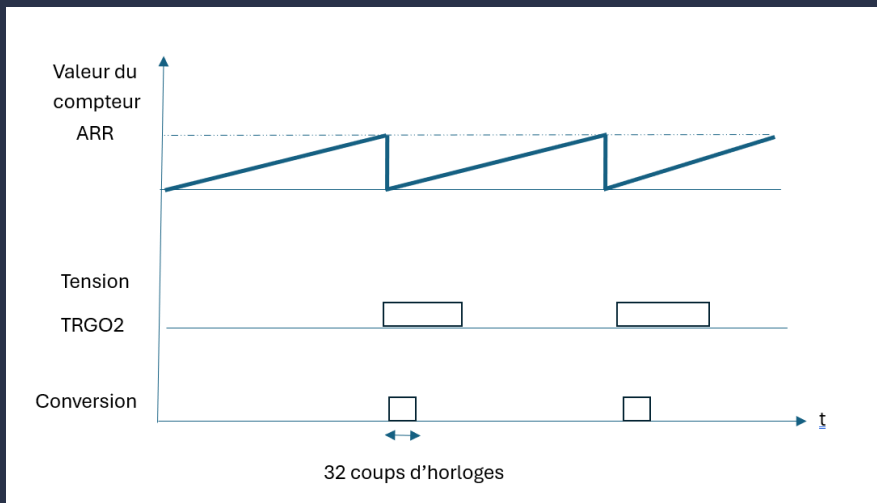
Deactivation of differential mode
Activation of single-ended mode
On pin 13 (pin PC3 pin of the Portenta)

```
CLEAR_BIT(ADC1->CFGR, ADC_CFGR_DISCEN_Msk);
```

Activation of discontinuous mode

```
SET_BIT(ADC1->CFGR, ADC_CFGR_EXTEN_0 | ADC_CFGR_EXTEN_1 | ADC_CFGR_EXTSEL_1 | ADC_CFGR_EXTSEL_3);
```

Triggering of conversion on each rising edge of TRGO2 (clock signal)



```
SET_BIT(ADC1->CFGR, ADC_CFGR_DMNGT_0 | ADC_CFGR_DMNGT_1)
```

Use of DMA in mode circular.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

14

DMA mode

```
SET_BIT(DMA1_Stream1->CR, DMA_SxCR_CIRC_Msk);
```

Activates circular

```
SET_BIT(DMA1_Stream1->CR, DMA_SxCR_DBM_Msk);
```

Activates double-buffer mode

```
DMA1_Stream1->M0AR = (uint32_t)&buff->M7toM4_1; (size 20)  
DMA1_Stream1->M1AR = (uint32_t)&buff->M7toM4_2;
```

Addresses of the two buffers

```
SET_BIT(DMA1_Stream1->CR, DMA_SxCR_MINC_Msk);
```

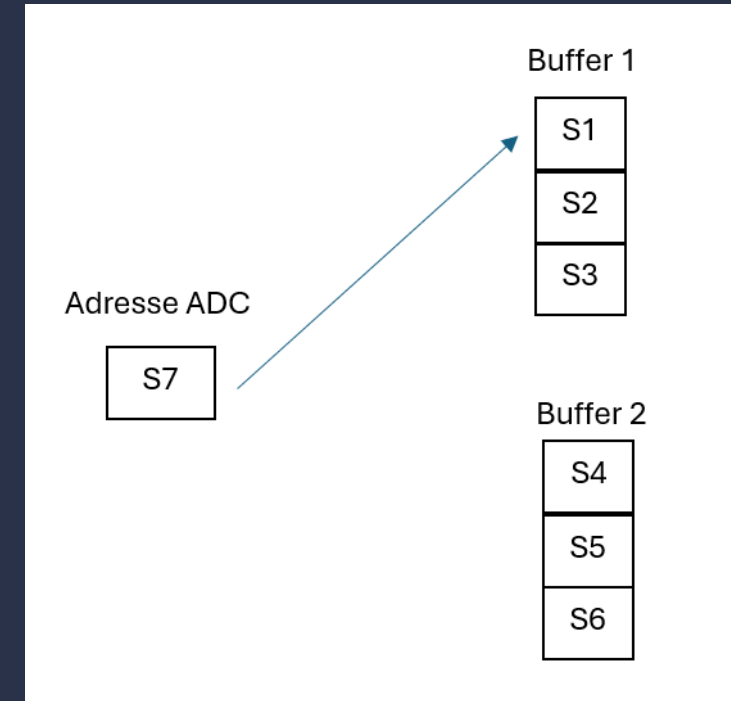
Activates automatic incrementing of the memory address during data transfer.

```
CLEAR_BIT(DMA1_Stream1->CR, DMA_SxCR_PINCOS_Msk);
```

Disables peripheral incrementing at the end of each transfer operation.

```
DMA1_Stream1->PAR = (uint32_t)&ADC1->DR;
```

Configures the source peripheral address for DMA transfer (ADC1)



Operation of circular double-buffer DMA

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

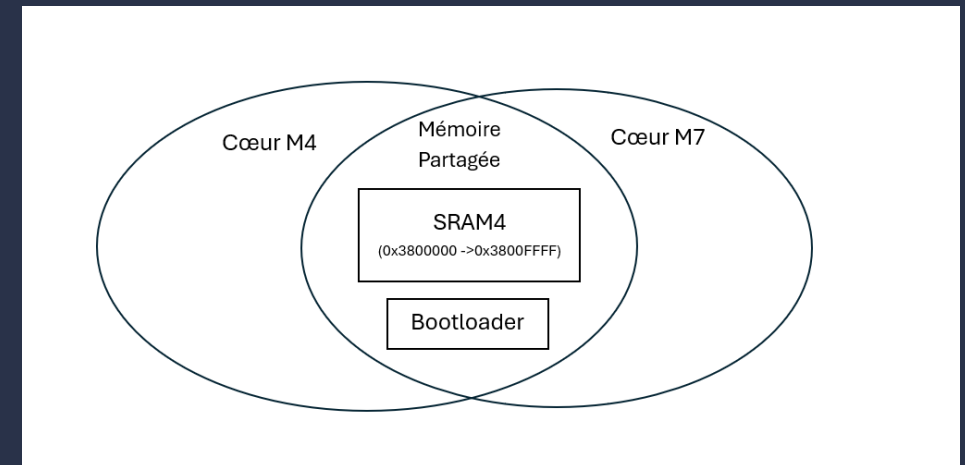
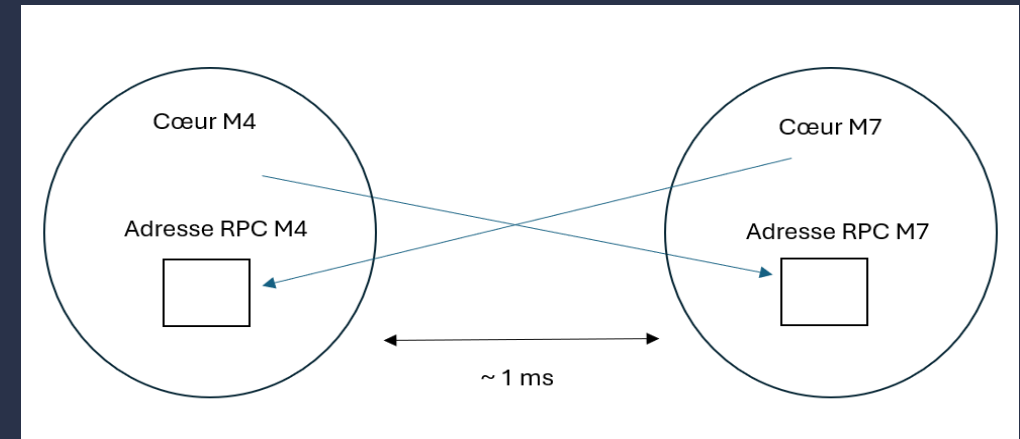
15

Communication Inter-core

Currently, all the data are on the M4 core, but to display them they must be transferred to the M7 core.

How should this data transfer be carried out between cores?

- RPC (Remote Procedure Call), a function implemented by Arduino, very easy to use but too slow, on the order of a millisecond.
- Use of shared memory, very complex because it is a completely handcrafted method with a risk of damaging the board, but very efficient and with no latency. It simply consists in creating the same data structure starting at the same shared-memory address on both processors.



```
volatile struct shared_data * const buff = (struct shared_data *)0x38001000;
```

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

16

M7 core - LabVIEW

This was a major problem in our predecessors' design. The same cable is used to power the board, upload scripts, and transfer data from the M7 core to LabVIEW. Thus, after uploading the code, the Arduino IDE has to be closed, the cable unplugged, then plugged back in, which is really not practical.

The M4 core has its own communication channel through the RX/TX pins. However, for coupling with the PC and LabVIEW It has to go through USB.

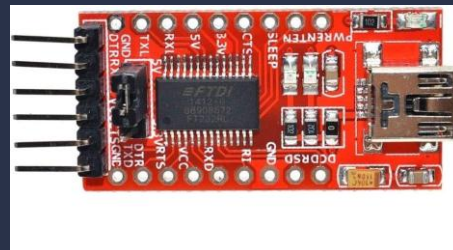
A solution?

Module FT232RL FTDI Type-C to TTL Serial Converter Adapter

This module can transmit the signal from the M4 core to the PC serial port (USB)



3 Mbps (max input rate)



12 Mbps (max output rate)

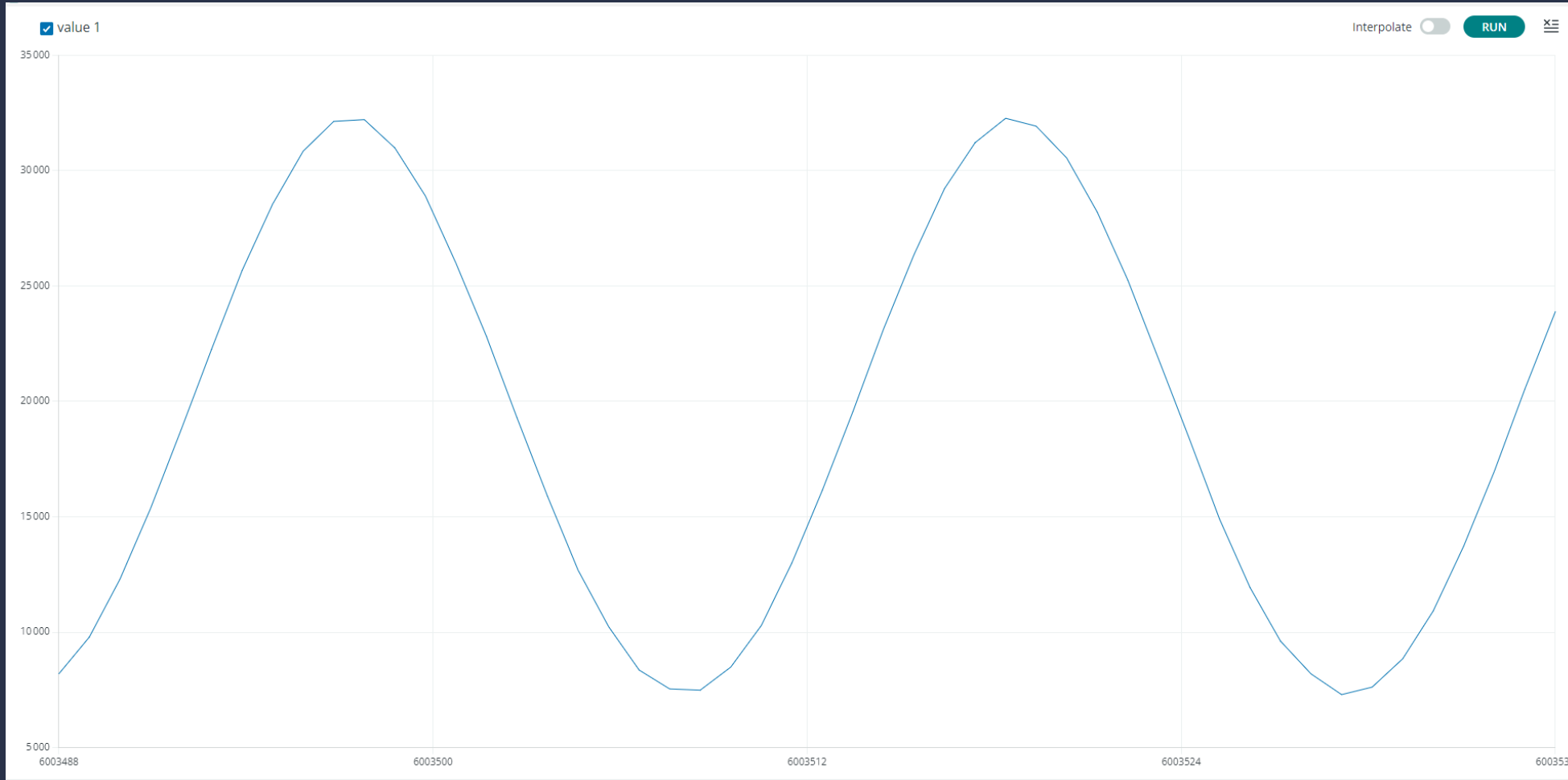


We need $200 \text{ kHz} * 16 \text{ bits} = 3.2 \text{ Mbps}$. We would therefore lose information.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

17

sampling of a sine wave at 10 kHz with the previous model.



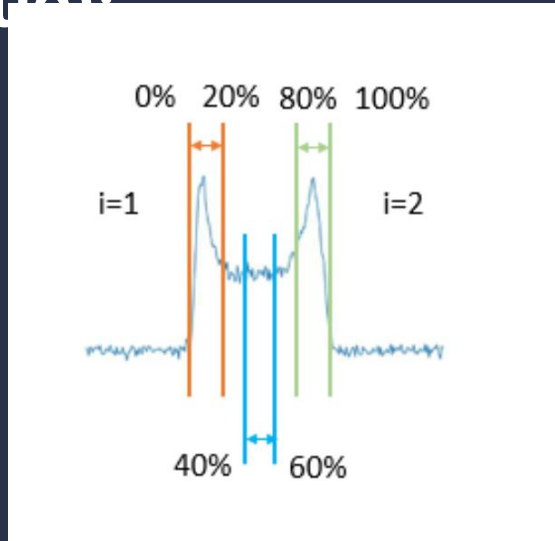
23 points per period

This supports the theory: $nb_{points \text{ par période}} = \frac{f_{\text{échantillonnage}}}{f_{\text{signal}}} = \frac{250 \text{ kHz}}{10 \text{ kHz}} \approx 25$

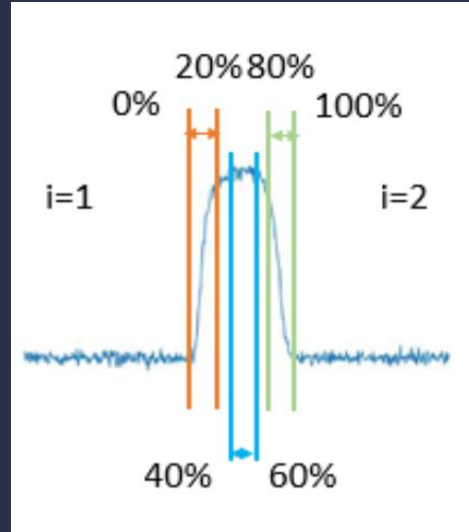
TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

18

Principle of the current sorting algorithm (SFR: simple fluorescence ratio):



"Rabbit ears" signal



"Dome" signal

t_g : taille de la goutte

$$SFR_1 = \frac{\int_0^{t_g^{*0,20}} fluo(t)dt}{\int_{t_g^{*0,40}}^{t_g^{*0,60}} fluo(t)dt}$$

Main advantage of this method:

- Enables real-time calculations. When the droplet is detected, it is enough to sum the converted values.
- Assuming the droplet is symmetrical (often the case), only half of the droplet needs to be processed.

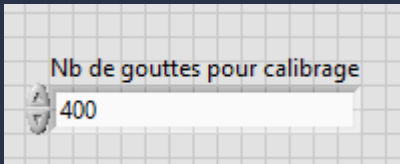
Constraint:

- A calibration must be performed to know the average distribution of droplet sizes.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

19

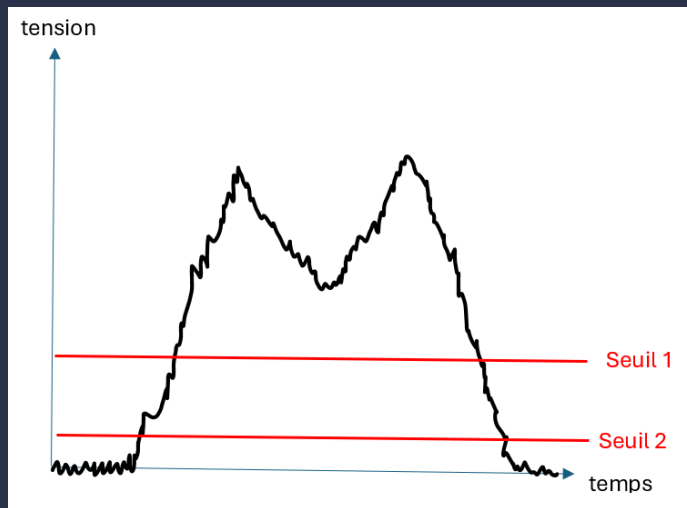
Obtaining droplet sizes (calibration)



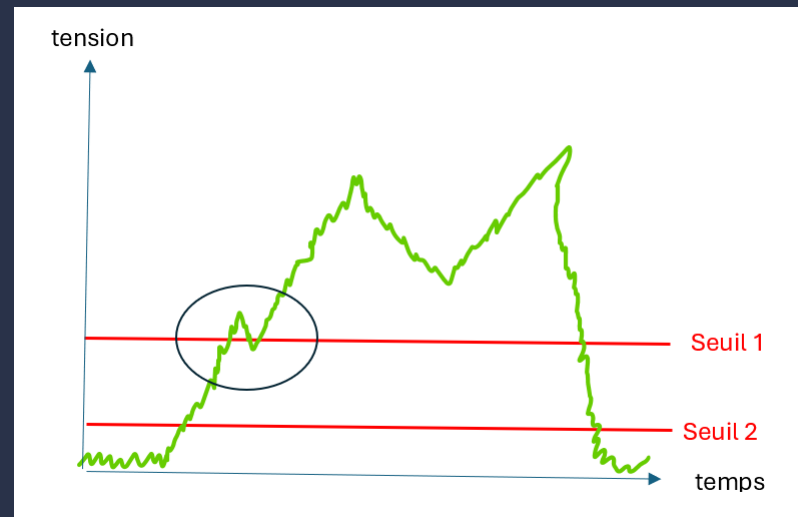
With LabVIEW, the user can configure the number of droplets to use for droplet-size calibration

Use of double thresholding:

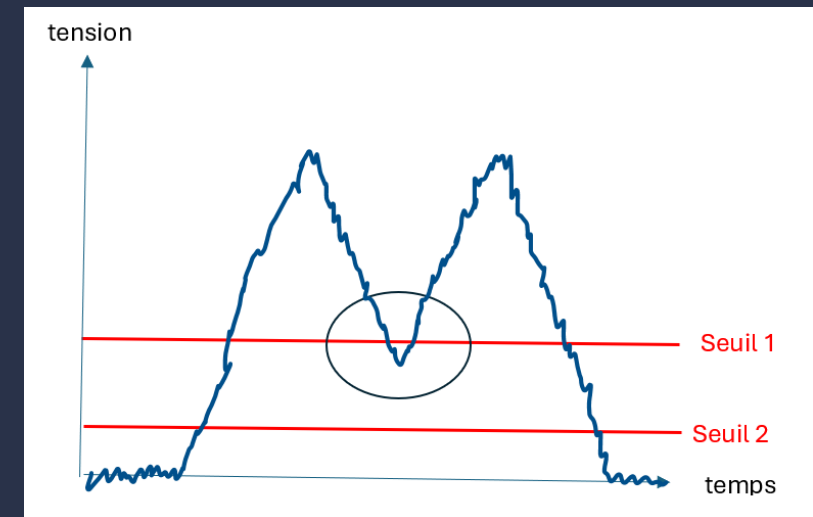
- When threshold 1 is exceeded, counting of the droplet size begins.
- When the signal falls below threshold 2 while counting is in progress, counting ends.



Classic case (no need for double thresholding)



Problematic case: recession around the threshold

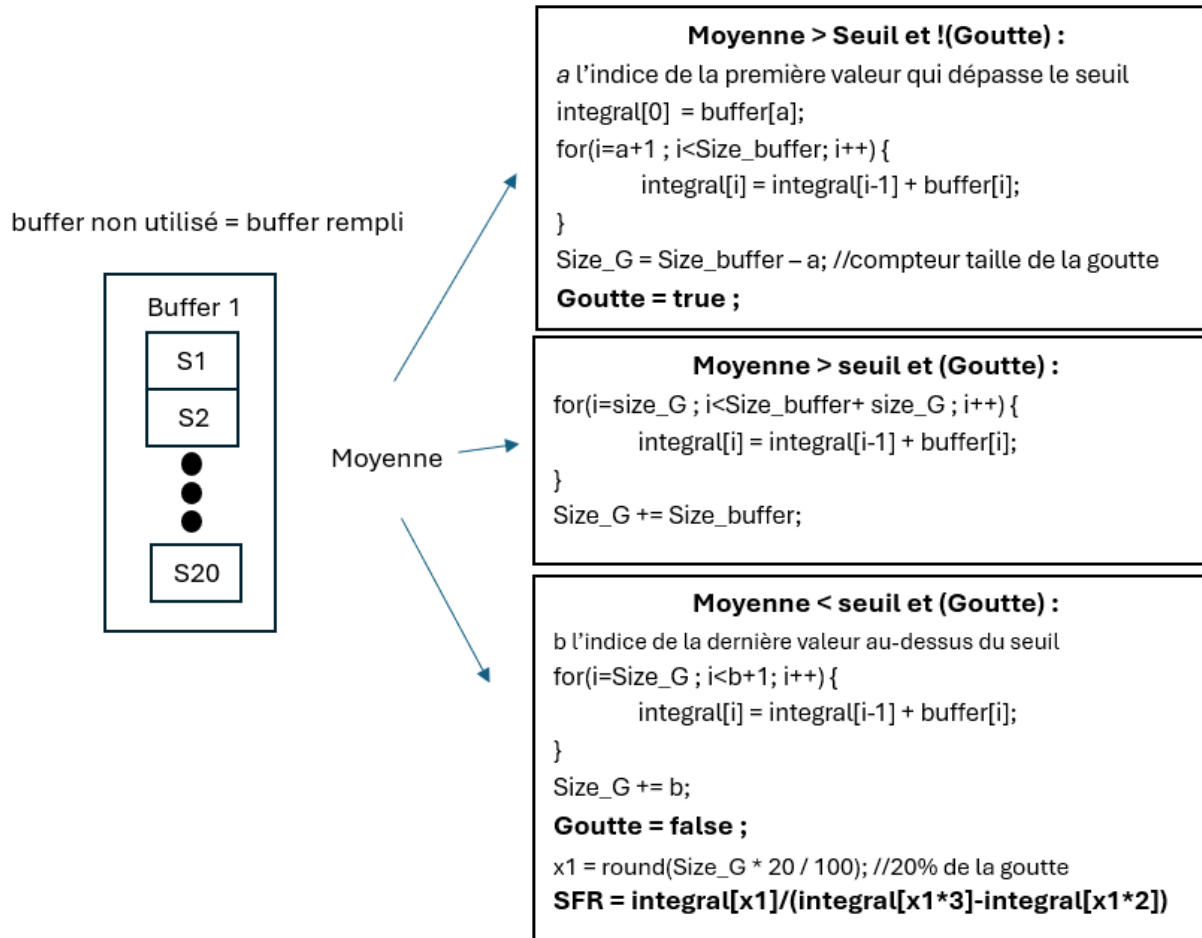


Problematic case: descent too abrupt

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

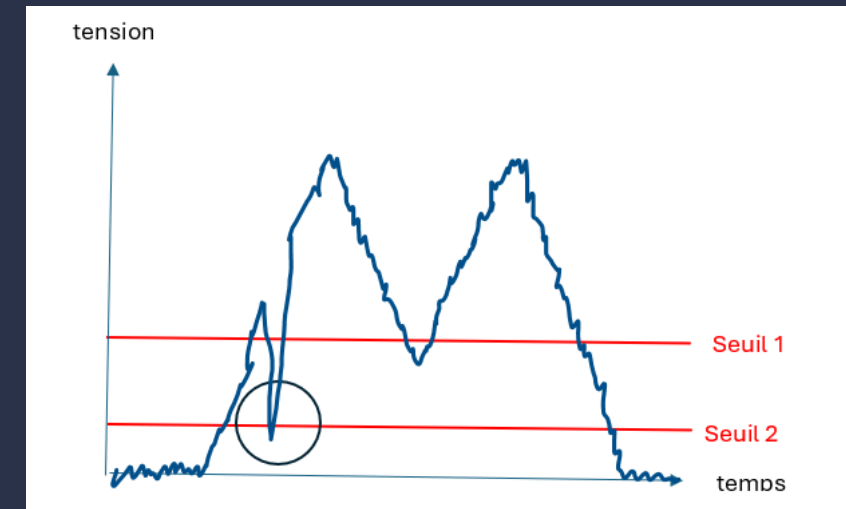
20

Sorting algorithm:



Advantage of the average:

- Very robust to fluctuations.



Drawback:

- The droplet size must be much larger than the buffer size.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

21

M7 core - LabVIEW:

The link between the M7 core and the PC uses the serial port (USB). The communication protocol is UART (Universal Asynchronous Receiver-Transmitter).

Flow problem:

We must be able to send the buffer data to LabVIEW before it is overwritten. As a reminder, this corresponds to 20 16-bit integers.

Filling time:

For this, we have the time required to fill the other buffer:

$$T_{remplissage} = 20 \times T_{ech} = \frac{20}{250 \text{ kHz}} = 80 \mu s$$

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

22

M7 core - LabVIEW:

Determining the right baud rate:

Hardware constraints impose communication in the following form:

(8 data bits, 1 start bit, 1 stop bit)

To optimize transmission time, we decompose 16-bit integers into 2 bytes, one most significant byte and one least significant byte

It will also be necessary to indicate to LabVIEW the type of data it is about to read by sending an ASCII character (8 bits) in the header.

```
val = 1234; //0000010011010010 (binary)
buffer[0] = val & 0xFF; //Least significant byte (11010010)
buffer[1] = (val >> 8) & 0xFF; //Most significant byte (00000100)
Serial.write(buffer, 2);
```

$$T_{vidange} = \frac{1 \text{ (bit de départ)} + 20 \text{ (taille de la liste)} * 2 * 8 \text{ (16 bits en 2 octets)} + 1 \text{ (bit de fin)} + (1 + 8 + 1) \text{ (en - tête)}}{\text{BaudRate (équivalent à un débit en bit)}}$$

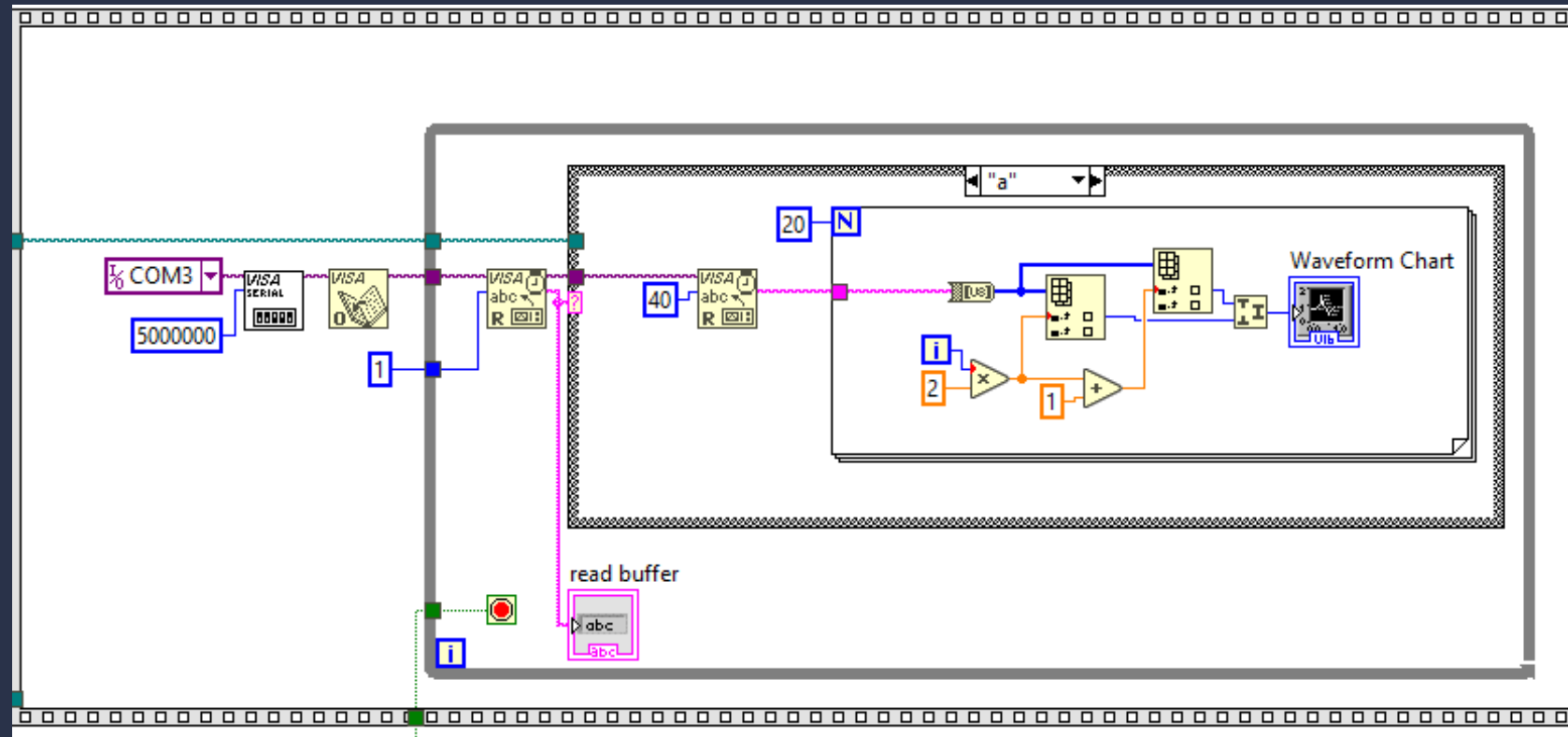
We can estimate that $T_{vidange} = 90\% \times T_{remplissage}$ is a sufficient margin to avoid risking data loss.

$$\text{BaudRate} = \frac{1 + 20 * 2 * 8 + 1 + 10}{T_{vidange}} = 4.7 \text{ MBaudRate} \approx 5 \text{ MBaudRate}$$

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

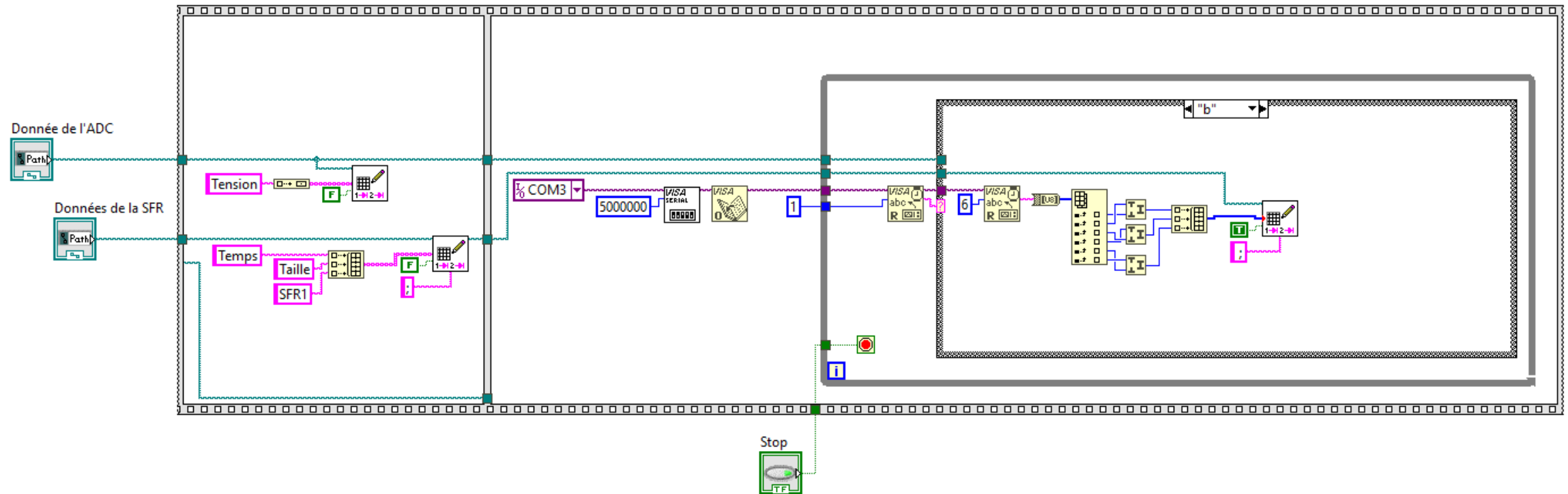
23

M7 core - LabVIEW:



24

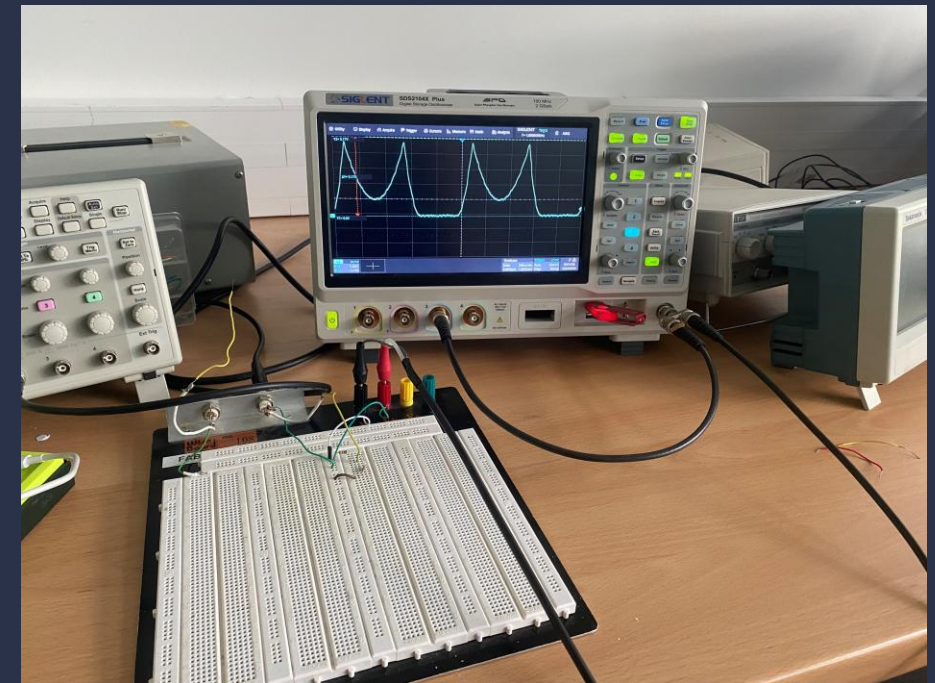
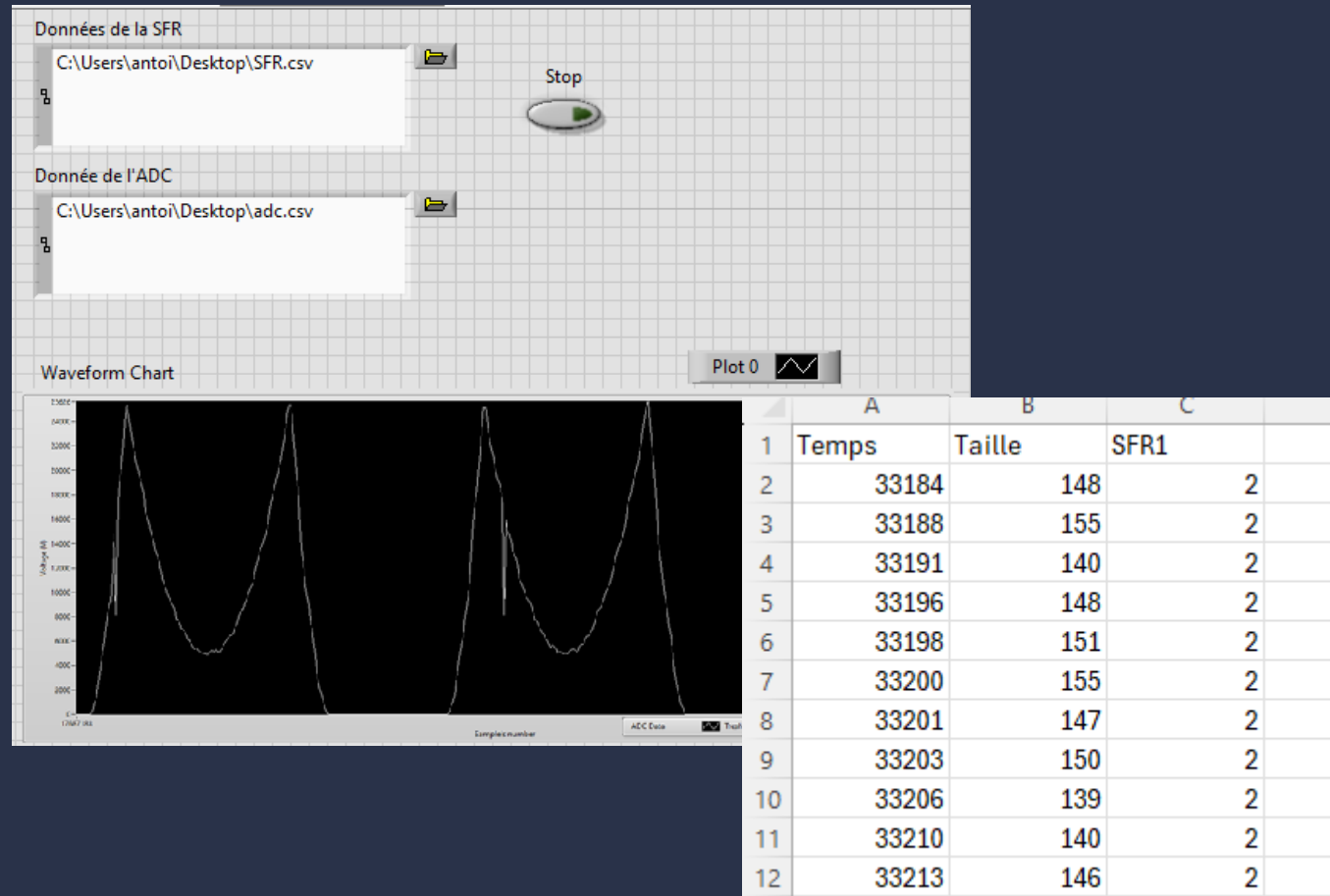
M7 core - LabVIEW:



TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

25

Final deliverable:



Generation of model signals:
slightly noisy "rabbit ears" at 1 kHz

The algorithm achieves 100% accuracy in recognizing model rabbit-ear signals.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

26

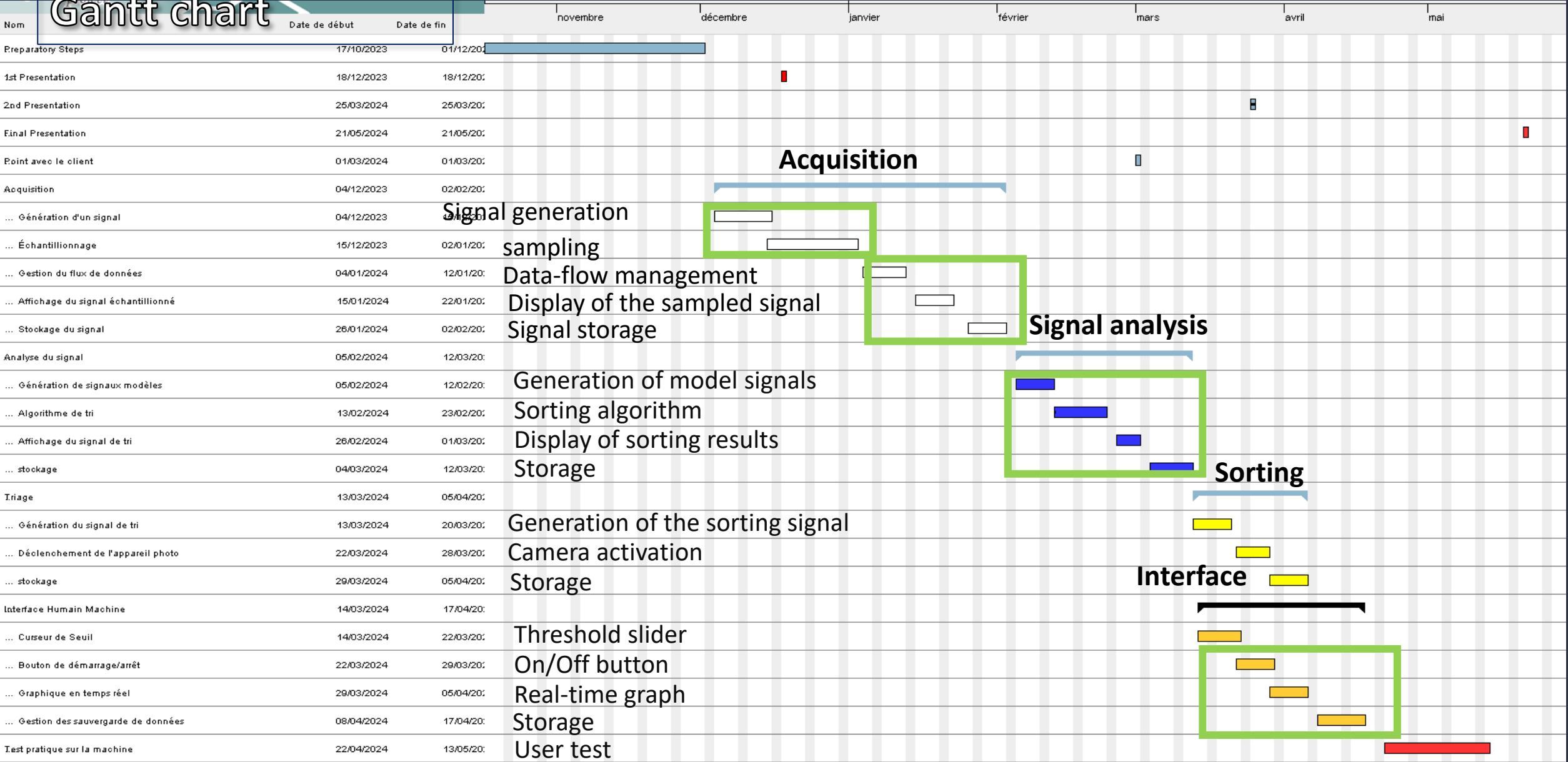
Improvement paths:

- It is clear that the board is not being pushed to its full potential; the ADC rate could be increased very substantially, but the communication protocol would probably have to be changed, moving toward Ethernet connectivity with much higher baud rates.
- Regarding the sorting algorithm, more sophisticated methods could be considered, such as using derivatives or the Fourier transform.

TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

27

Gantt chart



TI1A PROJECT: Embedded system (Arduino) programming and interface design for a high-throughput cell screening tool.

28

Deliverables:

- Arduino code (1,000 lines of annotated code)
- LabVIEW
- Short form describing how each part of the code works and the errors encountered.

Utilisation de la mémoire partagée :

Il suffit de déclarer exactement la même structure de donnée sur les deux cœurs en renseignant la même adresse de début située dans la mémoire partagée (ici 0x38001000)

Déclaration de la mémoire partagée :

Cœur M4	Cœur M7
<pre>#ifndef CORE_CM4 struct shared_data { double sfr1; volatile long tfinal; int DropSizeSFR; . . }; volatile struct shared_data * const buff = (struct shared_data *)0x38001000; #endif</pre>	<pre>#ifndef CORE_CM7 struct shared_data { double sfr1; volatile long tfinal; int DropSizeSFR; . . }; volatile struct shared_data * const buff = (struct shared_data *)0x38001000; #endif</pre>

Utilisation de la mémoire partagée : (M4 vers M7) :

Cœur M4	Cœur M7
<pre>#ifndef CORE_CM4 int taille_goutte = 150; buff->DropSizeSFR = taille_goutte; #endif</pre>	<pre>#ifndef CORE_CM7 int taille_goutte; taille_goutte = buff->DropSizeSFR; Serial.print(taille_goutte); #endif</pre>

Output Serial Monitor ×

Message (Enter to send message)

150

Thank you for your attention.

